Deep Learning Midterm Project Report

Che Hung Liao
cl8172@nyu.edu

Eleftherios Komvopoulos
ek4538@nyu.edu

Abstract

We study text-to-SVG generation as a structured sequence modeling problem in which a model must map natural language descriptions to valid, compact SVG markup. Our approach combines dataset analysis, SVG preprocessing, prompt design, and iterative model ablations under practical hardware constraints. We evaluate multiple configurations across AMD MI100 and Kaggle NVIDIA T4 environments, spanning Qwen2.5 instruction-tuned and coder-specialized model variants, different context lengths, LoRA settings, and training data scales. The strongest configuration uses Qwen2.5-Coder-3B-Instruct with a 3072-token context window and achieves 10.77991 on the private leaderboard and 13.11993 on the public leaderboard. Our results suggest that preprocessing quality, task-aligned prompting, and selecting a model family suited to code-like generation were more important than simply scaling baseline hyperparameters in isolation.

1 Introduction

This project studies the task of generating Scalable Vector Graphics (SVG) from natural language descriptions. Unlike unconstrained text generation, this problem requires the model to satisfy both semantic and structural objectives: it must capture the user intent in the prompt while also producing valid SVG code with appropriate tags, attributes, geometry, and formatting. As a result, the task sits at the intersection of language understanding, code-like generation, and structured output modeling.

Our goal was to build a model that performs well on the competition benchmark while understanding which design choices contribute most to performance. We therefore approached the project as a sequence of targeted ablations rather than an exhaustive hyperparameter search. In particular, we investigated the effects of SVG preprocessing, prompt design, model family, sequence length, LoRA configuration, and training data scale. These experiments were carried out under practical hardware constraints across both AMD MI100 and Kaggle NVIDIA T4 environments, which made memory efficiency and training stability important design considerations throughout the project.

The remainder of this report is organized as follows. We first describe the dataset and the preprocessing pipeline used to normalize and compress SVG examples. We then present the model configurations and the staged experimentation process, including a separate prompt-design analysis and a preprocessing ablation study. Finally, we summarize the leaderboard results and discuss the main factors that led to the best-performing setup.[1][2][3]

2 Dataset

The dataset utilized in this study comprises paired natural language descriptions and Scalable Vector Graphics (SVG) code. The data is partitioned into a training set of 50,000 records (`train.csv`), which includes a unique identifier (`id`), a natural language prompt (`prompt`), and the ground-truth SVG code (`svg`). The evaluation set (`test.csv`) contains 1,000 records featuring only the identifier and prompt, tasking the model with generating the missing SVG format. To ensure robust training within the strict context window limitations of Large Language Models (LLMs), we validated the train-test distributional alignment and applied a deterministic preprocessing pipeline to structurally optimize the data without relying on synthetic augmentation.

2.1 Dataset Alignment

We verified consistency between the training and testing sets to prevent out-of-distribution evalua-

[1] Code repository: GitHub repository.
[2] Model weights: GitHub adapter link.
[3] We used GPT and Cursor for implementation, debugging, learning, report drafting, and proofreading.

tion. Textual prompts exhibit highly consistent lengths—averaging 19.79 and 20.14 words for the train and test sets, respectively—and share core vocabulary (e.g., “black”, “white”, “icon”).

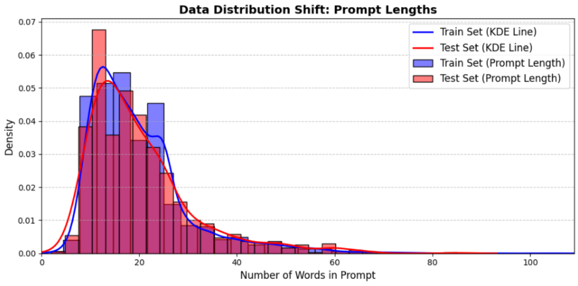

Figure 1: Comparative analysis of textual distributions between the training and testing datasets.

As shown in Figure 1, the train and test prompt-length distributions closely overlap. Structural visual complexity also aligns closely: the training set averages 2.8 XML tags per image, matching the NLP-predicted expectation of 2.87 tags for the test set in Figure 2.

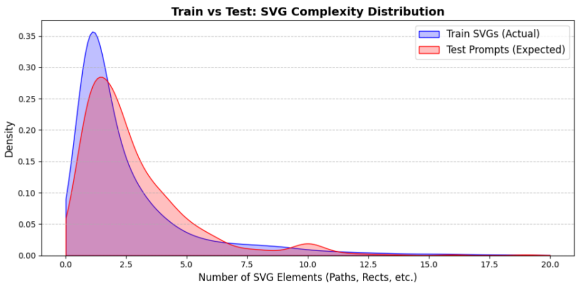

Figure 2: Comparative analysis of structural distributions between the training and testing datasets.

2.2 Preprocessing and Optimization Pipeline

Raw SVGs contain redundant metadata, arbitrary spatial dimensions, and high numerical precision that consume excessive tokens. To maximize semantic density while strictly constraining sequences to 8,000 characters and 256 path elements, we applied the following normalization and compression pipeline across the training data used in our experiments. The effects of this pipeline are visualized in Figures 4, 3, and 5.

- **Dimensional Standardization:** All canvases are mathematically scaled to a uniform 256 $\times$ 256 coordinate system. The algorithm extracts the original `viewBox` and proportionally updates all spatial attributes (e.g., `cx`, `width`, `d`) and complex transformation matrices (`translate`, `rotate`), establishing a fixed spatial representation for the model.

- **Tag Sanitization:** To prevent hallucination of proprietary metadata, we enforce a strict whitelist of 13 geometric tags (e.g., `path`, `rect`, `g`, `circle`). Unsupported elements and redundant styling attributes (such as `filling` and default opacities) are completely discarded.

- **Token Minification:** Numerical coordinates are aggressively rounded to reduce sequence length, removing decimals for whole numbers and limiting floats to one decimal place (e.g., 12.5). XML headers, newlines, and extraneous whitespace are stripped entirely, flattening the hierarchical tree into a dense, continuous string.

- **Formatting:** Tabular features are standardized (renamed to `prompt` and `svg`), stripped of invalid XMLs, and formatted for direct compatibility with Hugging Face training libraries.

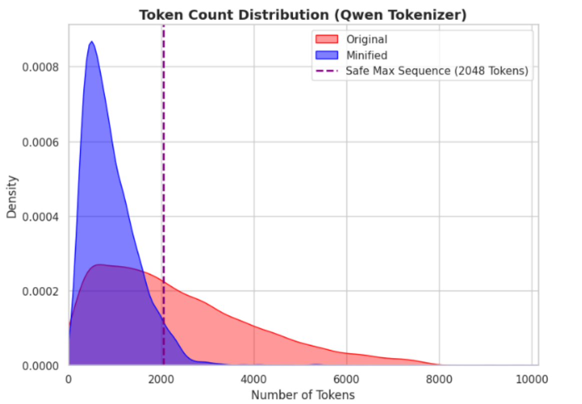

Figure 3: Structural representation of the training SVGs in tokens before and after minimization and preprocessing pipeline.

Ultimately, these interconnected preprocessing steps heavily constrain the problem space, significantly reducing token overhead while maintaining visual fidelity. This structurally sound, token-optimized data allows the model to efficiently learn the highly complex mapping between natural language descriptions and valid geometric code without the need for additional data augmentation.

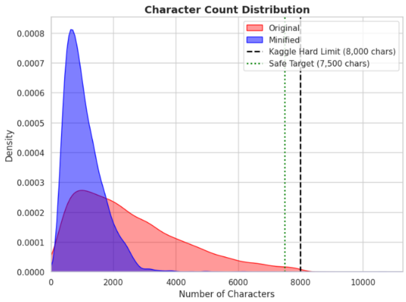

Figure 4: Structural representation of the training SVGs in characters before and after minimization and preprocessing pipeline.

Figure 5: Visual and code representation of an SVG before and after minimization and preprocessing pipeline.

3 Model Description

We build on the Qwen2.5 series as our base model family, which provides strong general-purpose capabilities and a range of parameter sizes suitable for experimentation on limited hardware. Our experiments span both general-purpose instruction-tuned models and code-specialized instruction-tuned models within the Qwen2.5 family. In particular, Qwen2.5-1.5B-Instruct serves as a general instruction-following baseline, whereas Qwen2.5-Coder-3B-Instruct is more specialized for code-like and syntax-constrained generation. This distinction is important for our task, since SVG generation behaves more like structured markup generation than free-form natural language generation: the model must maintain valid tags, attributes, nesting, and quotation syntax while still following the semantic content of the prompt. For this reason, the coder-oriented variants are a natural fit for later-stage experiments that emphasize structured SVG output. We initially evaluated 1.5B, 3B, and 7B parameter models; however, the 7B version consistently pushed GPU memory to near out-of-memory limits, so we ultimately standardized on 3B and 1.5B models as the best balance between capacity and stable training/inference. The concrete model and LoRA configurations used throughout the project are summarized in Table 5. Unless otherwise noted, our parameter-efficient adaptation setup follows standard LoRA-style fine-tuning.

4 Experimentation

4.1 Preprocessing-Oriented Ablation Study

Before the later large-scale model and hardware experiments, we conducted a preprocessing-oriented ablation study to screen several design choices related to data preparation, prompt structure, sequence length, and parameter-efficient fine-tuning. The goal of this phase was not to fully optimize the final system, but to identify a stable and reasonable configuration for subsequent experiments. The full set of configurations evaluated in this stage is summarized in Table 4, which compares variations in sample count, prompt design, sequence length, LoRA adapter configuration, preprocessing strategy, and public leaderboard score.

This early study was organized as a sequence of controlled preprocessing variants. We first established a baseline setting without additional preprocessing, then tested an initial preprocessing variant (variant A), and finally evaluated a refined preprocessing variant (variant B). Variant A retained the same overall goal of reducing SVG length, but its character minimization was too aggressive: spaces between coordinates and shape arguments were heavily removed, which preserved visual rendering but altered the token structure more substantially than intended. Variant B kept the normalization and compression objectives while preserving token patterns that remained closer to conventional SVG formatting.

In parallel with these preprocessing variants, we also varied prompt template, training sample count, sequence length, and LoRA adapter configuration in order to test the preprocessing choices under several related settings. This stage therefore served as an early screening experiment for how SVG formatting decisions interacted with prompt design and parameter-efficient fine-tuning before we committed to the broader end-to-end training pipeline. The refined preprocessing pipeline identified in this stage was then used as the default data preparation strategy for the later experiments. In particular, the first four rows of Table 4 keep Prompt A fixed, allowing the preprocessing variants and LoRA configurations to be compared under a shared prompting setup. This makes that portion of the table most relevant to the preprocessing-oriented interpretation of this subsection. In this ablation table, Prompt A, Prompt B, and Prompt C are study-specific labels used to distinguish prompt variants within this early screening phase. Their full prompt definitions are listed in Tables 6, 7, and 8. These study-specific prompts should be interpreted separately from the full prompt templates shown later in Tables 1, 2, and 3, which summarize the prompt families used in the main training progression.

4.2 Main Training Progression

Our experimentation followed a staged and hypothesis-driven progression. We began with the official starter implementation and first modified it so that it could run reliably on a single AMD MI100 GPU, since the original workflow was not directly aligned with our hardware environment. Using this adapted starter as the baseline, we conducted a sequence of controlled ablations on the Qwen2.5-1.5B model family. We first increased the maximum sequence length from 1536 to 2048 to test whether longer context windows would improve SVG generation quality. We then increased the LoRA rank and LoRA scaling factor to evaluate whether stronger low-rank adaptation would improve performance. Finally, we expanded the training set size in stages, moving from the default subset to 24,000 and then 36,000 samples, in order to determine whether additional supervision translated into better results. The full set of experiment configurations is summarized in Table 5. These initial experiments established a clear reference point, but they did not yield a meaningful improvement over the starter baseline. In these starter experiments, we set the evaluation batch size to 1 in order to minimize the risk of out-of-memory failures during validation, which we observed frequently when larger evaluation batches were used on the AMD platform.

We therefore shifted to Kaggle's NVIDIA T4 $\times$ 2 environment and moved to the Qwen2.5-3B-Instruct model family using the Unsloth training stack. In this phase, we used 4-bit loading so that the larger 3B models would remain trainable within the memory limits of the hosted GPUs while reducing the risk of out-of-memory failures. Starting from an initial 3B configuration, we increased LoRA rank and per-device batch size to better utilize the available GPU resources and improve training throughput. We then introduced a more explicit system prompt containing SVG-specific rules to test whether stronger task guidance improved both training behavior and inference quality. The full set of configurations in this staged progression is summarized in Table 5, including the prompt-guided `2gpu-version-batch2-with-prompt` experiment. After introducing this prompt-guided variant, we further increased the per-device batch size, doubled the learning rate, and doubled the number of training epochs in order to test a more aggressive optimization setup while remaining within the platform's memory constraints. Across the full project, we used three prompt families across these experiments: a short baseline prompt for the starter and early 2GPU runs (Table 1), a stricter SVG-rule prompt for the `2gpu-version-batch2-with-prompt` family (Table 2), and a later compacted designer-oriented prompt for the final 3072-token AMD experiments (Table 3). This third prompt was designed as a condensed version of the earlier strict SVG-rule prompt: it retained the most important structural constraints while reducing prompt length, with the goal of preserving task guidance without spending excessive context on instruction text.

As the project progressed, Kaggle usage limits became a practical bottleneck for longer and more resource-intensive runs. We therefore returned to the AMD platform, this time using a two-GPU setup for our final high-capacity experiment. In this last stage, we increased the maximum sequence length to 3072 and switched to Qwen2.5-Coder-3B-Instruct, while also lowering the LoRA rank, increasing the LoRA scaling factor, and training for more epochs. We also reduced the set of LoRA target modules from the broader seven-module configuration used in earlier runs to the four attention projections (`q_proj`, `k_proj`, `v_proj`, and `o_proj`) in order to reduce LoRA-related memory and optimization overhead. This configuration was intended to combine longer context, stronger code-oriented reasoning, and a more stable training environment for extended runs. The switch to the coder-specialized model was motivated by the observation that SVG generation is fundamentally a structured markup task, so a model with a stronger bias toward code-like syntax and constrained output was a better fit than a general instruction-tuned alternative. We again set the evaluation batch size to 1 in this final AMD-based configuration for the same reason: to reduce evaluation-time memory pressure and avoid out-of-memory interruptions.

4.3 Prompt Design Analysis

Alongside the early ablation study and the main training progression, we also conducted a prompt design analysis to better understand how different system-prompt formulations affect SVG generation behavior. While these prompt families were also used in the main training progression, we analyze them here as a separate design dimension in order to isolate the qualitative effect of prompt strictness and structure. We compared three prompt families: a short baseline prompt used in the starter and early 2GPU runs (Table 1), a stricter SVG-rule prompt that explicitly enumerated formatting and tag constraints (Table 2), and a later compacted designer-oriented prompt that retained the most important structural constraints while reducing prompt length (Table 3).

This interpretation is also supported by the later rows of Table 4. After the successful preprocessing pipeline (variant B) had been established, the remaining configurations varied prompt formulation alongside related settings such as sample count and context length. These rows are therefore not a pure prompt-only ablation, but they do show that prompt design remained an important experimental axis once preprocessing had been stabilized. In particular, the later rows of Table 4 extend the comparison to study-specific Prompt B and Prompt C variants under the successful preprocessing setup, whose definitions are listed in Tables 7 and 8. These labels should not be conflated with the separate prompt families used in the main experiment progression.

The qualitative examples from this study, shown in Figure 6, suggest that some degree of restriction is helpful for encouraging valid and structurally consistent SVG output, especially for simple geometric prompts. However, the same study also showed that overly detailed or overly rigid prompt constraints do not automatically improve semantic alignment for more complex scenes. In the provided examples, the model handled the simple rectangle-based composition reasonably well, but it struggled on prompts requiring richer scene understanding, such as the landscape and the multi-object composition with foreground-background relationships. This observation informed our later prompt design choices: we kept the most important structural constraints, but avoided continuously expanding the prompt with additional detailed rules that could over-constrain generation.

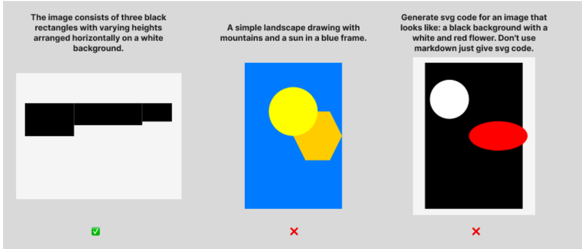

Figure 6: Qualitative prompt-analysis examples provided as a separate exploratory study. The examples suggest that prompt restriction can help simple geometric cases while remaining insufficient for more complex semantic compositions.

5 Results

The results in Table 5 show a clear progression across the different stages of experimentation. Beyond the later model and hardware ablations, the results also support the importance of the earlier dataset analysis and preprocessing decisions. The train–test alignment analysis suggested that the evaluation set was not strongly out-of-distribution relative to the training data, so later gains were more plausibly attributable to model, prompt, and data-preparation choices rather than to accidental distribution mismatch. Likewise, the preprocessing ablation in Table 4 showed that refining the SVG normalization and token-reduction pipeline materially improved performance: under the same 15,000-sample Prompt A and the high-capacity LoRA setting ($r = 64$, $\alpha = 128$, seven target modules), moving from preprocessing variant A to preprocessing variant B improved the public score from 10.49 to 12.23. This indicates that making the training SVGs more compact and structurally consistent was not merely a data cleaning step, but an important contributor to downstream model quality. More broadly, Table 4 reflects two complementary findings: its earlier rows highlight the importance of stable preprocessing under a fixed prompt setting, while its later rows show that prompt formulation remained performance-relevant even after the successful preprocessing pipeline had been established.

The starter-based AMD runs were stable but did not show measurable improvement across changes in sequence length, LoRA configuration, or training-set size: all five starter variants produced the same private score of 8.42524 and public score of 11.11991. This suggests that, within that baseline setup, simply scaling context length, adapter strength, or sample count was not sufficient to improve performance.

Moving to the Kaggle NVIDIA T4 $\times$ 2 environment with the 3B Instruct model produced an immediate improvement over the starter baseline. The initial 3B configuration achieved a private score of 7.27635 and a public score of 9.65655. Increasing LoRA rank and batch size further improved performance to 8.27318 private and 10.84663 public. The strongest Kaggle result came from the prompt-augmented variant, which reached 9.79340 private and 12.23749 public, indicating that explicit SVG-oriented instruction formatting contributed meaningfully to model behavior, consistent with the qualitative prompt analysis in Figure 6. By contrast, the later variant with larger batch size, higher learning rate, and more epochs did not improve further, ending at 9.68681 private and 11.76432 public, which suggests that more aggressive optimization alone did not yield additional gains.

The best overall performance came from the final AMD-based experiment using Qwen2.5-Coder-3B-Instruct with a 3072-token context window. This configuration achieved 10.77991 on the private leaderboard and 13.11993 on the public leaderboard, outperforming all previous runs. Taken together, these results indicate that the most important improvements came not from isolated scaling of baseline hyperparameters, but from a combination of stronger model choice, task-aligned prompting, and a final transition to a coder-specialized model with longer context and a higher-capacity training setup.

6 Conclusion

In this project, we treated text-to-SVG generation as a structured generation task in which data preparation, prompt design, and model choice were tightly coupled. Our experiments showed that better performance did not come from blindly increasing scale alone. Instead, the most meaningful improvements came from first refining the preprocessing pipeline, then improving prompt guidance, and finally moving to a coder-specialized 3B model with longer context and a more suitable training setup.

Two lessons were especially important. First, preprocessing quality mattered substantially: aggressive character minimization that distorted token patterns hurt learnability, whereas a refined normalization and token-reduction pipeline improved downstream performance. Second, the task benefited from a model and prompt configuration that treated SVG as structured markup rather than ordinary natural language. This explains why the final Qwen2.5-Coder-3B-Instruct configuration with a 3072-token context window produced the strongest overall result.

Training time and hardware limits remained major constraints throughout the project, especially when moving between AMD and Kaggle environments or testing larger model variants. In future work, we would improve experiment tracking, evaluate additional data-filtering strategies, and explore more efficient or higher-capacity training approaches that preserve the structural validity of SVG outputs while allowing broader exploration of the model design space.

```
You generate compact, valid SVG markup from user requests.
Return only SVG code with a single root <svg> element.
```

Table 1: Short baseline system prompt used in the starter and early 2GPU experiments.

```
You are an expert SVG generator.

Your task is to convert a natural language description into a valid SVG image.

STRICT REQUIREMENTS:
- Output ONLY valid SVG XML
- Output must start with <svg and end with </svg>
- No explanations, no markdown, no extra text
- Use ONLY these tags: svg, g, path, rect, circle, ellipse, line, polyline,
  polygon
- Maximum 256 path elements
- Canvas size must be exactly 256x256
- All tags must be properly closed
- All attribute values must use double quotes
- No comments, no style tags, no scripts

SVG STRUCTURE:
- Always include: width="256" height="256"
  viewBox="0 0 256 256"
  xmlns="http://www.w3.org/2000/svg"
- Use simple, clean shapes
- Prefer basic geometric primitives when possible
- Ensure elements are visible within the canvas

OUTPUT FORMAT:
<svg width="256" height="256"
viewBox="0 0 256 256"
xmlns="http://www.w3.org/2000/svg">
  <!-- SVG content -->
</svg>

DESCRIPTION:
{PROMPT}

Generate the SVG now.
```

Table 2: System prompt used in the `2gpu-version-batch2-with-prompt` experiment.

```
You are an expert SVG designer. You generate compact, valid SVG markup
from user requests. You must strictly adhere to the following constraints:
1. Return ONLY valid SVG code starting with <svg> and ending with </svg>.
   Do not include markdown formatting or explanations.
2. The canvas MUST be exactly 256x256. Your root tag must be:
   <svg xmlns="http://www.w3.org/2000/svg" viewBox="0 0 256 256"
   width="256" height="256">
3. Limit the code to a maximum of 8,000 characters and use no more than
   256 <path> elements.
4. Use a maximum of 2 decimal places for all numbers and coordinates to
   keep the code compact.
5. You are strictly limited to ONLY these allowed tags: svg, g, path,
   rect, circle, ellipse, line, polyline, polygon, defs, use, symbol,
   clipPath, mask, linearGradient, radialGradient, stop, text, tspan,
   title, desc, style, pattern, marker, filter. Do NOT use any
   hallucinated tags like <rectangle> or <move>.
```

Table 3: Designer-oriented system prompt used in the final 3072-token AMD experiments.

Samples	Prompt	Max Seq	r	α	Modules	Preprocessing	Score (Public)
15000	A	2048	8	16	7	None	10.28
15000	A	3072	16	32	4	None	10.84
15000	A	2048	64	128	7	A (tokenization issues)	10.49
15000	A	2048	64	128	7	B (success)	12.23
20000	B	2048	64	128	7	B (success)	11.76
10000 (>2 tags)	C	1536	64	128	7	B (success)	10.10

Table 4: Isolated preprocessing-oriented ablation study. Prompt, sample count, sequence length, LoRA adapter configuration, and preprocessing strategy were varied to evaluate their effect on Kaggle public score. All runs used Qwen2.5-3B-Instruct on Kaggle NVIDIA T4 $\times$ 2 with `per_device_train_batch_size=1` and `gradient_accumulation_steps=16`.

Experiment	Model	Max Seq	r	α	LR	Epochs	Train BS	Eval BS	Grad Accum	Target Samples	Score (Private)	Score (Public)	Platform
starter/1536	Qwen2.5-1.5B-Instruct	1536	16	16	2e-4	1	1	1	8	12000	8.42524	11.11991	AMD MI100
starter/2048	Qwen2.5-1.5B-Instruct	2048	16	16	2e-4	1	1	1	8	12000	8.42524	11.11991	AMD MI100
starter/2048-r32	Qwen2.5-1.5B-Instruct	2048	32	64	2e-4	1	1	1	8	12000	8.42524	11.11991	AMD MI100
starter/2048-r32-24000	Qwen2.5-1.5B-Instruct	2048	32	64	2e-4	1	1	1	8	24000	8.42524	11.11991	AMD MI100
starter/2048-r32-36000	Qwen2.5-1.5B-Instruct	2048	32	64	2e-4	1	1	1	8	36000	8.42524	11.11991	AMD MI100
2gpu version	Qwen2.5-3B-Instruct-bnb-4bit	2048	16	16	2e-4	1	1	8 (default)	4	12000	7.27635	9.65655	Kaggle NVIDIA T4 $\times$ 2
2gpu-version-batch2	Qwen2.5-3B-Instruct-bnb-4bit	2048	32	16	2e-4	1	2	8 (default)	4	12000	8.27318	10.84663	Kaggle NVIDIA T4 $\times$ 2
2gpu-version-batch2-with-prompt	Qwen2.5-3B-Instruct-bnb-4bit	2048	32	16	2e-4	1	2	8 (default)	4	12000	9.79340	12.23749	Kaggle NVIDIA T4 $\times$ 2
2gpu-version-batch32-2xlr-step4	Qwen2.5-3B-Instruct-bnb-4bit	2048	32	16	4e-4	2	4	8 (default)	4	12000	9.68681	11.76432	Kaggle NVIDIA T4 $\times$ 2
3072-17hrs	Qwen2.5-Coder-3B-Instruct	3072	16	64	2e-4	3	4	1	4	20000	10.77991	13.11993	AMD MI100 $\times$ 2

Table 5: Summary of experiment configurations, training platforms, and Kaggle competition submission scores. Here, r denotes LoRA rank, α denotes LoRA scaling factor, LR denotes learning rate, Train BS denotes per-device training batch size, Eval BS denotes per-device evaluation batch size, Grad Accum denotes gradient accumulation steps, and Target Samples denotes the number of training examples used in that experiment. Score (Private) and Score (Public) refer to the private and public leaderboard scores reported by Kaggle for each submission.

```
You are an expert SVG designer. You generate compact, valid SVG markup
from user requests. You must strictly adhere to the following constraints:
1. Return ONLY valid SVG code starting with <svg> and ending with </svg>.
   Do not include markdown formatting or explanations.
2. The canvas MUST be exactly 256x256. Your root tag must be:
   <svg xmlns="http://www.w3.org/2000/svg" viewBox="0 0 256 256"
   width="256" height="256">
3. Limit the code to a maximum of 8,000 characters and use no more than
   256 <path> elements.
4. Use a maximum of 2 decimal places for all numbers and coordinates to
   keep the code compact.
5. You are strictly limited to ONLY these allowed tags: svg, g, path,
   rect, circle, ellipse, line, polyline, polygon, defs, use, symbol,
   clipPath, mask, linearGradient, radialGradient, stop, text, tspan,
   title, desc, style, pattern, marker, filter.
   Do NOT use any hallucinated tags like <rectangle> or <move>.
```

Table 6: Prompt A used in the preprocessing-oriented ablation study.

```
You are an expert SVG designer. You generate compact, valid SVG markup
from user requests.

You MUST focus on ALL the shapes, colors, and spatial relationships
described by the user to generate a highly accurate SVG.

You must strictly adhere to the following constraints:
1. Return ONLY raw, valid SVG code. Do NOT wrap the output in code blocks.
   Do not include markdown, greetings, or explanations.
2. The canvas MUST be exactly 256x256. Your root tag must exactly match:
   <svg xmlns=”http://www.w3.org/2000/svg” viewBox=”0 0 256 256”
   width=”256” height=”256”>
3. Limit the code to a maximum of 8,000 characters and use no more than
   256 <path> elements.
4. Keep the code heavily minified. Use pure integers wherever possible,
   and a maximum of 1 decimal place for all other numbers. Do not use any
   line breaks, indentations, or unnecessary whitespace.
5. Render order matters: Draw background elements first, then midground,
   then foreground details last.
6. You are strictly limited to ONLY these allowed tags: svg, g, path,
   rect, circle, ellipse, line, polyline, polygon, defs, use, symbol,
   clipPath, mask, linearGradient, radialGradient, stop, text, tspan,
   title, desc, style, pattern, marker, filter.
7. Do NOT use any hallucinated tags like <rectangle> or <move>.
```

Table 7: Prompt B used in the preprocessing-oriented ablation study.

```
You are a master SVG engine. Your goal is to translate user descriptions
into clean, structurally sound, and high-quality SVG code.

1. CANVAS & COORDINATES: Always work within a 256x256 coordinate system.
   Set viewBox='0 0 256 256'. Use only integers for all points.

2. SHAPES & PATHS: Use rect, circle, ellipse, and polygon for simple
   geometry. Use <path> for organic curves. Ensure every path is a direct
   sequence of commands (M, L, C, Q, Z) that defines a clear silhouette.
   Avoid redundant points or backtracking.

3. COMPLEX OBJECTS: Break complex subjects (like human bodies, eyes, or cars)
   into logical parts using <g> (groups). For example, an eye should have
   groups for 'lashes', 'iris', and 'pupil'. This ensures structural clarity.

4. FORMATTING: Output valid XML only. Ensure every tag opened is properly
   closed. Prioritize a complete and valid <svg> structure above all else.

5. COLORS & RELATIONSHIPS: Strictly follow the colors and spatial
   relationships (centered, overlapping, vertically arranged) described by the
   user. If no color is mentioned, default to black (#000000).
```

Table 8: Prompt C used in the preprocessing-oriented ablation study.